

A. DIVIDE AND CONQUER

The idea is simple: **Break a big problem into smaller sub-problems**, solve each sub-problem independently, then **combine the results**. This is the "Divide → Conquer → Combine" strategy.

CONCEPTS

1. Master Method

$$T(n) = aT(n/b) + \theta(n^k \log^p n)$$

Where $a \geq 1$, $b > 1$, $k \geq 0$ and p is a real no.

- 1) If $a > b^k$, then $T(n) = \theta(n \log_b a)$
- 2) If $a = b^k$ then
 - a. If $p > -1$, then $T(n) = \theta(n \log_b a \log^{p+1} n)$
 - b. If $p = -1$, then $T(n) = \theta(n \log_b a \log \log n)$
 - c. If $p < -1$, then $T(n) = \theta(n^{\log_b a})$
- 3) If $a < b^k$ then
 - a. If $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$
 - b. If $p < 0$, then $T(n) = O(n^k)$

Quick Tip: Compare $f(n)$ with $n^{(\log_b a)}$. Whichever grows faster dominates the complexity.

2. Recursion Tree Method

Draw the recursion as a **tree**. At each level, calculate the work done. Sum up all levels to get total time.

Steps: (1) Draw the tree (2) Find cost at each level (3) Count total levels (4) Sum all costs

Example

$$T(n) = 2T(n/2) + n$$

Level 0: n | Level 1: $n/2 + n/2 = n$ | Level 2: n | ... | Level $\log_2 n$: n

Total levels = $\log_2 n + 1 \rightarrow$ Total cost = $n \times \log_2 n \rightarrow T(n) = \theta(n \log n)$

3. Substitution Method

You **guess the solution**, then **prove it by Mathematical Induction**.

Steps: (1) Guess $T(n) = O(f(n))$ (2) Assume $T(k) \leq c \cdot f(k)$ for all $k < n$ (3) Prove it holds for n

IMPORTANT ALGORITHMS

① Binary Search

Search for a target in a **sorted array** by repeatedly **halving the search space**.

Pre-condition: [Array must be sorted](#)

 Steps

1. Set low = 0, high = n-1
2. Find mid = (low + high) / 2
3. If arr[mid] == target → Found! Return mid
4. If arr[mid] < target → Search right half: low = mid + 1
5. If arr[mid] > target → Search left half: high = mid - 1
6. Repeat until low > high (not found)

Metric	Best Case	Average Case	Worst Case
Time	$O(1)$	$O(\log n)$	$O(\log n)$
Space	$O(1)$	$O(1)$	$O(1)$

Recurrence: $T(n) = T(n/2) + O(1) \rightarrow$ Solves to **$O(\log n)$**

Data Structure Used: [Array \(sorted\)](#) | **Key Property:** Divide search space in half each step

② Merge Sort

A classic **stable sorting algorithm**. Split array into halves, sort each, then merge them in order.

 Steps

1. If array has 1 element → already sorted, return
2. Divide: Split array into left half and right half
3. Conquer: Recursively sort left half and right half
4. Combine: Merge two sorted halves into one sorted array
→ Compare elements from each half one by one, pick the smaller

Metric	Best Case	Average Case	Worst Case
Time	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Space	$O(n)$	$O(n)$	$O(n)$

Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow$ Solves to **$O(n \log n)$**

Data Structure: [Array / Linked List](#) | **Stable:** YES | **In-Place:** NO (needs extra $O(n)$ space)

 MCQ Tip

Merge Sort is the BEST sorting algorithm for Linked Lists (no random access needed).
It always runs in $O(n \log n)$ — no worst case like Quick Sort's $O(n^2)$.

③ Quick Sort

Pick a **pivot**, place it in its correct position such that all smaller elements are to its left, all larger to its right. Repeat for sub-arrays.

📋 Steps (Lomuto Partition)

1. Choose pivot (usually last element)
2. $i = \text{low} - 1$ (pointer for smaller elements)
3. For j from low to high-1:
→ If $\text{arr}[j] \leq \text{pivot}$: increment i , swap $\text{arr}[i]$ and $\text{arr}[j]$
4. Swap $\text{arr}[i+1]$ and pivot → pivot is now in correct position
5. Recursively sort left part and right part

Metric	Best Case	Average Case	Worst Case
Time	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Space	$O(\log n)$	$O(\log n)$	$O(n)$

Worst Case occurs when: Array is already sorted or reverse sorted and pivot is always first/last

Pivot Strategies: First | Last | Random | Median of Three

Data Structure: Array | **Stable:** NO | **In-Place:** YES

💡 MCQ Tip

Quick Sort has $O(n^2)$ worst case but is faster in practice than Merge Sort due to cache locality. Randomized Quick Sort reduces worst case probability significantly.

④ Strassen Matrix Multiplication

Normal matrix multiplication of two $n \times n$ matrices takes $O(n^3)$. Strassen reduces it using only **7 recursive multiplications** instead of 8.

📋 Key Idea

Divide each $n \times n$ matrix into four $(n/2) \times (n/2)$ sub-matrices.
Compute 7 products (M_1 to M_7) using additions/subtractions.
Combine M_1 – M_7 to get the four quadrants of the result matrix.
Recurrence: $T(n) = 7T(n/2) + O(n^2)$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n^{2.807})$
Space	-	-	$O(n^2)$

Key Formula: $\log_2 7 \approx 2.807 \rightarrow$ Strassen: $O(n^{2.807})$ vs Normal: $O(n^3)$

MCQ Note: Strassen uses 7 multiplications and 18 additions per level. Practical only for very large matrices.

⑤ Closest Pair of Points

Find the pair of points with the **minimum Euclidean distance** from a set of n points in 2D plane.

 Steps

1. Sort points by x-coordinate
2. Divide: Split into left half and right half by median x
3. Conquer: Recursively find closest pair in each half → get d_L and d_R
4. $d = \min(d_L, d_R)$
5. Check strip: points within distance d of the dividing line
6. For each strip point, check at most 7 other points → find d_{strip}
7. Answer = $\min(d, d_{\text{strip}})$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n \log n)$
Space	-	-	$O(n)$

⑥ Karatsuba Multiplication

Fast algorithm to multiply two large numbers. Normal long multiplication takes $O(n^2)$. Karatsuba uses only **3 recursive multiplications** instead of 4.

 Key Idea

Split each n -digit number into two halves: high and low

$$x = x_{\text{hi}} \times 10^{(n/2)} + x_{\text{lo}} ; y = y_{\text{hi}} \times 10^{(n/2)} + y_{\text{lo}}$$

Compute: $A = x_{\text{hi}} \times y_{\text{hi}}$, $C = x_{\text{lo}} \times y_{\text{lo}}$

$$B = (x_{\text{hi}} + x_{\text{lo}})(y_{\text{hi}} + y_{\text{lo}}) - A - C$$

$$\text{Result} = A \times 10^n + B \times 10^{(n/2)} + C$$

$$\text{Recurrence: } T(n) = 3T(n/2) + O(n)$$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n^{1.585})$
Space	-	-	$O(n)$

Key: $\log_2 3 \approx 1.585 \rightarrow$ Karatsuba: $O(n^{1.585})$ vs Normal: $O(n^2)$

B. DYNAMIC PROGRAMMING (DP)

DP is an optimization technique that **stores results of sub-problems** to avoid re-computation. It works when a problem has **Overlapping Sub-problems** + **Optimal Substructure**.

| CONCEPTS

1. Memoization (Top-Down)

Solve using **recursion** but **store each computed result** in a table (usually a HashMap or array). Before computing, **check if already solved**.

Analogy: Like writing answers on a sticky note so you don't re-solve the same sum.

Pros & Cons

- ✓ Easy to implement (just modify recursive code)
- ✓ Only computes needed sub-problems (lazy evaluation)
- ✗ Recursive overhead, stack space used

2. Tabulation (Bottom-Up)

Solve **iteratively** by filling a table from the **smallest sub-problems upward**, until the final answer is built.

Analogy: Like building a house — lay the foundation (base cases) first, then build up floor by floor.

Pros & Cons

- ✓ No recursion stack overflow risk
- ✓ Faster in practice (better cache performance)
- ✗ May compute sub-problems not needed

3. State Space

The **set of all possible sub-problem configurations**. Each **state** is a unique combination of parameters. Example: In Knapsack, state = (item index, remaining capacity)

4. State Transition

The **recurrence relation** that shows how to compute a state from smaller states. The core of DP formulation.

Example: Fibonacci: $F(n) = F(n-1) + F(n-2)$ → each state depends on two previous states

| IMPORTANT DP PROBLEMS

① Fibonacci Numbers

Problem: Find the nth Fibonacci number where $F(n) = F(n-1) + F(n-2)$, $F(0)=0$, $F(1)=1$

ALGORITHM DESIGN TECHNIQUES

Approach	Time	Space
Naive Recursion	$O(2^n)$	$O(n)$
Memoization	$O(n)$	$O(n)$
Tabulation	$O(n)$	$O(n)$
Space Optimized	$O(n)$	$O(1)$

DP Table: $dp[i] = dp[i-1] + dp[i-2]$ for $i \geq 2$, $dp[0]=0$, $dp[1]=1$

② Matrix Chain Multiplication (MCM)

Given n matrices, find the **optimal parenthesization** that minimizes total scalar multiplications.

Key Insight: Multiplying matrix $A(p \times q)$ with $B(q \times r)$ costs $p \times q \times r$ operations

Steps

Let $dp[i][j]$ = min cost to multiply matrices from i to j

Base case: $dp[i][i] = 0$ (single matrix needs no multiplication)

Transition: $dp[i][j] = \min$ over all k from i to $j-1$ of:

$$dp[i][k] + dp[k+1][j] + p[i-1] \times p[k] \times p[j]$$

Answer: $dp[1][n]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n^3)$
Space	-	-	$O(n^2)$

Data Structure: 2D DP Table ($n \times n$)

③ 0/1 Knapsack

Given items with **weights and values**, and a bag of capacity W . Maximize total value. Each item can be **taken (1) or left (0)** — no fractions.

Recurrence

$dp[i][w]$ = max value using first i items with capacity w

If $weight[i] > w$: $dp[i][w] = dp[i-1][w]$ (can't pick item i)

Else: $dp[i][w] = \max(dp[i-1][w], value[i] + dp[i-1][w - weight[i]])$

Answer: $dp[n][W]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n \times W)$
Space	-	-	$O(n \times W)$ or $O(W)$

Data Structure: 2D DP Table | **Note:** This is a **Pseudo-polynomial** algorithm (depends on W)

MCQ Tip

0/1 Knapsack $\rightarrow$ DP (items can't be split)

Fractional Knapsack $\rightarrow$ Greedy (items can be split)

④ Coin Change Problem

Given coin denominations and a target amount, find the **minimum number of coins** to make the amount.



Recurrence

$dp[i]$ = min coins needed to make amount i
 $dp[0] = 0$ (base case)
 $dp[i] = \min \text{ over all coins } c \text{ where } c \leq i: dp[i - c] + 1$
 Answer: $dp[\text{amount}]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(\text{amount} \times n)$
Space	-	-	$O(\text{amount})$

⑤ Rod Cutting

Given a rod of length n and price table for each length, find the **maximum revenue** by cutting the rod.



Recurrence

$dp[i]$ = max revenue from rod of length i
 $dp[0] = 0$
 $dp[i] = \max \text{ over } j \text{ from } 1 \text{ to } i: \text{price}[j] + dp[i - j]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n^2)$
Space	-	-	$O(n)$

⑥ Longest Common Subsequence (LCS)

Find the length of the **longest subsequence** common to two strings. (Subsequence = not necessarily contiguous)



Recurrence

$dp[i][j]$ = LCS length of first i chars of X and first j chars of Y
 If $X[i] == Y[j]$: $dp[i][j] = dp[i-1][j-1] + 1$
 Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
 Answer: $dp[m][n]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(m \times n)$
Space	-	-	$O(m \times n)$ or $O(\min(m, n))$

Data Structure: [2D DP Table](#) | MCQ: LCS $\neq$ LCS of reversed string (that's LPS)

⑦ Longest Increasing Subsequence (LIS)

Find the length of the **longest strictly increasing subsequence** in an array.

 Recurrence (DP approach)

dp[i] = length of LIS ending at index i
 dp[i] = max(dp[j] + 1) for all j < i where arr[j] < arr[i]
 dp[i] = 1 if no such j exists
 Answer: max of all dp[i]

Metric	Best Case	Average Case	Worst Case
Time (DP)	-	-	$O(n^2)$
Time (Binary Search)	-	-	$O(n \log n)$
Space	-	-	$O(n)$

⑧ Bellman-Ford Algorithm

Find **shortest path from a single source** to all other vertices. Works with **negative weight edges**. Detects **negative weight cycles**.

 Steps

1. Initialize dist[source] = 0, all others = ∞
2. Repeat (V-1) times:
 For every edge (u, v, w): If dist[u] + w < dist[v] → update dist[v]
3. Negative Cycle Check: Do one more relaxation.
 If any distance still updates → negative cycle exists

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(V \times E)$
Space	-	-	$O(V)$

Data Structure: [Edge List](#) | **Key Difference from Dijkstra:** Bellman-Ford **handles negative weights**, Dijkstra does NOT

⑨ Floyd-Warshall Algorithm

Find **shortest paths between ALL pairs of vertices**. Works with negative weights (but no negative cycles).

 Recurrence

dist[i][j] = shortest distance from i to j
 Initialize: dist[i][j] = weight of edge (i,j), or ∞ if no edge, 0 if i==j
 For each intermediate vertex k from 1 to V:
 For each i, j: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(V^3)$
Space	-	-	$O(V^2)$

Data Structure: [2D Distance Matrix \(V×V\)](#)

⑩ Edit Distance (Levenshtein Distance)

Minimum number of **insertions, deletions, replacements** to convert string X to string Y.



Recurrence

$dp[i][j]$ = min edits to convert first i chars of X to first j chars of Y

If $X[i] == Y[j]$: $dp[i][j] = dp[i-1][j-1]$ (no operation needed)

Else: $dp[i][j] = 1 + \min(dp[i-1][j], \leftarrow \text{Delete}$

$dp[i][j-1], \leftarrow \text{Insert}$

$dp[i-1][j-1]) \leftarrow \text{Replace}$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(m \times n)$
Space	-	-	$O(m \times n)$ or $O(\min(m,n))$

⑪ Traveling Salesman Problem (DP with Bitmask)

Visit all n cities exactly once and return to start with **minimum total distance**. DP uses **bitmask** to represent visited cities.



State Definition

$dp[mask][i]$ = min cost to visit cities in 'mask', ending at city i

mask is a bitmask where bit j = 1 means city j is visited

Transition: $dp[mask][i] = \min \text{ over } j \text{ not in mask:}$

$dp[\text{mask without } i][j] + \text{dist}[j][i]$

Answer: min over all i of $dp[(1 < n) - 1][i] + \text{dist}[i][0]$

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(2^n \times n^2)$
Space	-	-	$O(2^n \times n)$

Much better than Brute Force $O(n!)$ but still exponential — feasible only for small n (≤ 20)

IMPORTANT TERMS

Term	Meaning
Cost Table / DP Table	The table that stores solutions to sub-problems
Bottom-Up	Start from smallest sub-problems, build up to the answer (Tabulation)
Top-Down	Start from the main problem, recurse, store results (Memoization)
Overlapping Sub-problems	Same sub-problems are solved multiple times (key DP property)
Optimal Substructure	Optimal solution of problem = made of optimal solutions of sub-problems

C. GREEDY ALGORITHMS

A greedy algorithm makes the **locally optimal choice** at each step, hoping to find the **globally optimal solution**. No backtracking. Works only when **Greedy Choice Property** + **Optimal Substructure** hold.



Greedy vs DP

Greedy: Makes ONE choice per step, never reconsiders → Faster but not always correct
 DP: Considers ALL choices, stores results → Slower but correct for more problems

① Activity Selection Problem

Select the **maximum number of non-overlapping activities**. Each activity has a start and finish time.



Steps

1. Sort activities by their FINISH TIME (ascending)
2. Select the first activity
3. For each next activity: if its start time $\geq$ finish time of last selected → select it
4. Repeat until all activities are considered

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n \log n)$
Space	-	-	$O(1)$

Key: Always pick the activity that finishes earliest | **Greedy Choice:** Earliest finish time first

② Fractional Knapsack

Same as 0/1 Knapsack but items **can be taken in fractions**. Select items by **value/weight ratio** (value per unit weight).



Steps

1. Calculate value/weight ratio for each item
2. Sort items by ratio in DECREASING order
3. Greedily pick items: take as much as possible of highest ratio item first
4. If bag is full or no items left → stop

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n \log n)$
Space	-	-	$O(1)$



MCQ Tip

Fractional Knapsack → Greedy gives OPTIMAL solution
 0/1 Knapsack → Greedy may NOT give optimal; use DP

③ Huffman Coding

A **lossless data compression** algorithm. Assigns **shorter codes to more frequent characters** and longer codes to less frequent ones.

Steps

1. Count frequency of each character
2. Create a min-heap (priority queue) of leaf nodes
3. Repeat until heap has 1 node:
 - a. Extract two nodes with LOWEST frequency (min-heap)
 - b. Create new internal node with frequency = sum of two
 - c. Insert new node back into heap
4. The remaining node is the root of Huffman Tree
5. Assign 0 to left branches, 1 to right branches → read leaf paths = code

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n \log n)$
Space	-	-	$O(n)$

Data Structure: [Min-Heap \(Priority Queue\) + Binary Tree](#)

Property: Huffman codes are **prefix-free** (no code is prefix of another) → unambiguous decoding

④ Kruskal's Algorithm (Minimum Spanning Tree)

Find MST by **adding edges in increasing weight order**, skipping edges that form a cycle. Uses **Union-Find** data structure.

Steps

1. Sort ALL edges by weight (ascending)
2. Initialize: each vertex is its own set (Union-Find)
3. For each edge (u, v, w) in sorted order:
 - If u and v are in DIFFERENT sets → Add edge to MST, Union their sets
 - If same set → Skip (would form a cycle)
4. Stop when MST has (V-1) edges

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(E \log E)$ or $O(E \log V)$
Space	-	-	$O(V + E)$

Data Structure: [Edge List + Union-Find \(Disjoint Set Union\)](#) | Works well for **sparse graphs**

⑤ Prim's Algorithm (Minimum Spanning Tree)

Grow MST one vertex at a time. Always add the **minimum weight edge** that connects a visited vertex to an unvisited one.

Steps

1. Start with any vertex, mark it visited
2. Add all edges from visited vertices to a priority queue
3. Pick edge with MINIMUM weight that connects to unvisited vertex
4. Add that vertex to MST, mark visited
5. Repeat until all vertices are in MST

Metric	Best Case	Average Case	Worst Case
--------	-----------	--------------	------------

ALGORITHM DESIGN TECHNIQUES

Time (Adj Matrix)	-	-	$O(V^2)$
Time (Min-Heap)	-	-	$O(E \log V)$
Space	-	-	$O(V + E)$

Data Structure: [Min-Heap \(Priority Queue\) + Adjacency List](#) | Works well for **dense graphs**



Kruskal vs Prim

Kruskal: sorts edges globally → better for SPARSE graphs

Prim: grows from a vertex → better for DENSE graphs

Both give MST with SAME total weight (may differ in structure)

⑥ Dijkstra's Algorithm (Single Source Shortest Path)

Find shortest paths from **one source to all vertices**. Works only with **non-negative edge weights**.



Steps

1. Initialize $\text{dist}[\text{source}] = 0$, all others = ∞
2. Put all vertices in a min-heap (priority queue)
3. While heap is not empty:
 - a. Extract vertex u with MINIMUM distance
 - b. For each neighbor v of u :
If $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$ → Update $\text{dist}[v]$, re-add to heap

Metric	Best Case	Average Case	Worst Case
Time (Adj Matrix)	-	-	$O(V^2)$
Time (Min-Heap)	-	-	$O((V + E) \log V)$
Space	-	-	$O(V)$

Data Structure: [Min-Heap \(Priority Queue\)](#) | **DOES NOT** work with negative weights — use Bellman-Ford instead

⑦ Job Sequencing with Deadlines

Schedule jobs to **maximize total profit**. Each job has a deadline and profit. Job takes 1 unit of time.



Steps

1. Sort jobs by PROFIT in DECREASING order
2. Find max deadline D . Create slot array of size D , all empty
3. For each job (in sorted order):
Assign it to the latest available slot $\leq$ its deadline
4. Jobs assigned = optimal schedule

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(n^2)$ simple $O(n \log n)$ with Union-Find
Space	-	-	$O(n)$

D. BACKTRACKING

Backtracking is a **systematic trial-and-error** technique. Build the solution step by step; if a step violates constraints, **backtrack (undo) and try the next option**. It is essentially a **DFS on the State Space Tree**.

| CONCEPTS

1. State Space Tree

A tree that represents **all possible configurations** (states) of the solution. The **root** is the starting state; **leaves** are complete solutions or dead ends. Backtracking explores this tree using DFS.

2. DFS Search

Backtracking uses **Depth-First Search** to explore the state space tree. Go as deep as possible, backtrack when stuck.

3. Constraint Satisfaction

At each step, check if the **current partial solution satisfies all constraints**. If not → backtrack. This **prunes large portions of the search space**.

| IMPORTANT TERMS

Term	Meaning
Promising Node	A node that CAN lead to a valid solution (constraints satisfied so far)
Dead Node	A node that CANNOT lead to a valid solution → prune this branch
Feasible Solution	A complete solution that satisfies ALL constraints
Live Node	A node generated but not yet expanded

| PROBLEMS

① N-Queens Problem

Place N queens on an N×N chessboard so that **no two queens attack each other**. No two queens can share the same row, column, or diagonal.

Steps

1. Place queens one row at a time (row by row)
2. For each row, try each column (0 to N-1)
3. Check if placing queen at (row, col) is SAFE:
 - No queen in same column
 - No queen on same diagonal ($|row1 - row2| == |col1 - col2|$)

ALGORITHM DESIGN TECHNIQUES

4. If safe: place queen, move to next row (recurse)
5. If not safe OR no valid col: BACKTRACK (remove queen, try next col)

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(N!)$
Space	-	-	$O(N)$

For $N=8$ (classic): 92 distinct solutions exist

② Graph Coloring

Assign **colors to vertices** such that **no two adjacent vertices** share the same color, using **at most M colors**.

Steps

1. Start from vertex 0
2. Try assigning each color (1 to M)
3. Check if color is SAFE (no adjacent vertex has same color)
4. If safe: assign color, recurse to next vertex
5. If no safe color found: BACKTRACK

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(M^V)$
Space	-	-	$O(V)$

Chromatic Number: The **minimum M** needed to color the graph. Finding it is **NP-Hard**

③ Hamiltonian Cycle

Find a cycle that visits **every vertex exactly once** and returns to the start. (Compare: **Eulerian Circuit** visits every **edge** once)

Steps

1. Start from vertex 0
2. Try adding an unvisited adjacent vertex to path
3. If the last vertex in path has an edge back to vertex 0 AND all vertices visited → FOUND
4. If no valid next vertex → BACKTRACK

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(N!)$
Space	-	-	$O(N)$

NP-Complete problem — no polynomial algorithm known for general graphs

④ Subset Sum

Find a **subset of numbers** that adds up to a given **target sum S**.

Steps

1. At each step, decide: INCLUDE current element OR EXCLUDE it

2. If current sum == S → Found a solution
3. If current sum > S OR no elements left → BACKTRACK
4. Two branches per element → Binary tree structure

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(2^n)$ worst case
Space	-	-	$O(n)$

⑤ Sudoku Solver

Fill a 9×9 grid with digits 1-9 such that each row, column, and 3×3 box contains all digits 1-9.



Steps

1. Find an empty cell
2. Try digits 1 to 9
3. Check if digit is VALID (not in same row, column, or 3×3 box)
4. If valid: place digit, recurse to next empty cell
5. If no valid digit OR recursion fails → set cell to empty, BACKTRACK

Metric	Best Case	Average Case	Worst Case
Time	-	-	$O(9^{(n \times n)})$ worst case, fast in practice
Space	-	-	$O(n \times n)$

E. BRANCH AND BOUND

Branch and Bound is used to solve **optimization problems**. Like Backtracking, it explores a state space tree, but uses a **bounding function** to **prune branches that cannot lead to a better solution** than the best found so far.



Key Difference: Backtracking vs Branch and Bound

Backtracking → used for FEASIBILITY problems (find any valid solution)

Branch and Bound → used for OPTIMIZATION problems (find BEST solution)

B&B uses COST/BOUND to prune; Backtracking uses CONSTRAINTS to prune

CONCEPTS

Term	Definition
Bounding Function	A function that gives an upper/lower bound on best possible solution from current node. Used to prune unpromising branches.
Live Node	A node that has been generated but whose children have NOT yet been explored
E-node (Expanding Node)	The CURRENT node being expanded (its children are being generated)
Dead Node	A node that will NOT be expanded further (either pruned or fully explored)

PROBLEMS

① Traveling Salesman Problem (TSP)

Find the **shortest tour** visiting all cities exactly once and returning to start. B&B uses **lower bound** (e.g., **reduced cost matrix**) to prune branches where the minimum possible tour cost exceeds the current best.

Metric	Best Case	Average Case	Worst Case
Time (worst)	-	-	$O(n!)$ but B&B prunes heavily in practice
Space	-	-	$O(n^2)$ for cost matrix

② 0/1 Knapsack (B&B)

Find max value subset within weight W . B&B uses an **upper bound** (from fractional knapsack solution of remaining items) to prune branches.



Bounding Strategy

Upper Bound = current value + best possible value from remaining items

(Use fractional knapsack on remaining items for upper bound)

If upper bound $\leq$ current best solution → PRUNE this branch

ALGORITHM DESIGN TECHNIQUES

Metric	Best Case	Average Case	Worst Case
Time (worst)	-	-	$O(2^n)$ but pruning makes it much faster in practice
Space	-	-	$O(n)$

③ Assignment Problem

Assign n workers to n jobs minimizing total cost. Each worker assigned to exactly one job. B&B explores assignment tree, using **Hungarian algorithm-based bound**.

Metric	Best Case	Average Case	Worst Case
Time (Hungarian only)	-	-	$O(n^3)$
Time (B&B best case)	-	-	Much less than $O(n!)$
Space	-	-	$O(n^2)$

STRATEGIES

Strategy	Data Structure Used	Order of Exploration	Best For
FIFO Branch and Bound	Queue (BFS order)	Level by level (BFS)	Finding solution at lowest depth
LIFO Branch and Bound	Stack (DFS order)	Deep first (DFS)	Finding any feasible solution quickly
Least Cost Branch and Bound	Min-Heap (Priority Queue)	Node with LOWEST bound first	Finding OPTIMAL solution fastest (best pruning)



Most Important for Exams

Least Cost B&B = Best First Search → generally gives optimal solution with least exploration
FIFO B&B = BFS B&B → explores level by level, good for finding shortest path solutions
LIFO B&B = DFS B&B → uses less memory but may explore deeper unnecessarily

QUICK COMPARISON TABLE

Technique	Strategy	Works For	Time Complexity	Handles -ve weights
Divide & Conquer	Split → Solve → Merge	Sorting, Searching	Varies ($O(n \log n)$ typical)	N/A

ALGORITHM DESIGN TECHNIQUES

Dynamic Programming	Overlapping subproblems + Optimal substructure	Optimization, Counting	Polynomial (varies)	Yes (Bellman-Ford)
Greedy	Local optimal choice at each step	Optimization (when greedy choice prop holds)	$O(n \log n)$ typical	No (Dijkstra)
Backtracking	DFS + Constraint pruning	Feasibility / All solutions	Exponential worst case	N/A
Branch and Bound	DFS/BFS + Bound pruning	Optimization (exact)	Exponential worst, fast in practice	N/A

| MCQ QUICK REFERENCE

⚡ HIGH-FREQUENCY EXAM FACTS

- Binary Search: $O(\log n)$ | Array must be SORTED
- Merge Sort: $O(n \log n)$ always | Stable | NOT in-place | Best for Linked Lists
- Quick Sort: $O(n^2)$ worst (sorted input) | NOT stable | In-place
- Strassen: $O(n^{2.807})$ | 7 multiplications | Better than $O(n^3)$
- Karatsuba: $O(n^{1.585})$ | 3 multiplications | Better than $O(n^2)$
- Bellman-Ford: $O(VE)$ | Handles NEGATIVE weights | Detects negative cycles
- Dijkstra: $O((V+E) \log V)$ | NO negative weights | Greedy
- Floyd-Warshall: $O(V^3)$ | ALL pairs | 2D matrix DP
- Kruskal: $O(E \log E)$ | Edge-based | Union-Find | Sparse graphs
- Prim: $O(E \log V)$ | Vertex-based | Min-Heap | Dense graphs
- Huffman: $O(n \log n)$ | Min-Heap | Prefix-free codes
- 0/1 Knapsack: DP $\rightarrow O(nW)$ | Fractional Knapsack: Greedy $\rightarrow O(n \log n)$
- LCS: $O(mn)$ | LIS: $O(n^2)$ DP or $O(n \log n)$ with Binary Search
- TSP: DP $\rightarrow O(2^n n^2)$ | NP-Hard (no poly algorithm known)
- N-Queens: Backtracking | 8-Queens has 92 solutions
- Hamiltonian Cycle: NP-Complete | Eulerian Circuit: $O(E)$ via DFS